Tecnológico de Estudios Superiores de Ecatepec

División de Ingeniería en Sistemas Computacionales

ACTIVIDAD 1

Evolución de las aplicaciones web

Institución	Tecnológico de Estudios Superiores de Ecatepec
Carrera	División de Ingeniería en Sistemas Computacionales
Materia	Programación Web
Nombre	Marcos Joaquín García Villanueva
Nombre del Profesor	Alejandra González Sánchez
Grupo	5501
Actividad	1 — Evolución de las aplicaciones web

Introducción

Las aplicaciones web constituyen hoy una de las plataformas de software más importantes del mundo. Prácticamente cualquier servicio digital que utilizamos a diario —correo electrónico, banca en línea, comercio electrónico, redes sociales, plataformas de transmisión de video o sistemas empresariales— se apoya, total o parcialmente, en tecnologías web. Sin embargo, este nivel de madurez no apareció de un día para otro: es el resultado de más de tres décadas de evolución técnica, en las que cada generación de herramientas resolvió las limitaciones de la anterior y abrió nuevas posibilidades.

Comprender esta evolución no es un ejercicio meramente histórico. Conocer de dónde vienen las tecnologías actuales permite entender por qué funcionan como lo hacen, qué problemas fueron diseñadas para resolver y cuándo conviene utilizar cada una. Para un desarrollador que debe preparar un ambiente de desarrollo moderno, este conocimiento es la base para tomar decisiones de arquitectura informadas en lugar de seguir modas pasajeras.

En el contexto de PropertyStream Tech, empresa del sector PropTech responsable de plataformas como Smart Listings y RentStream, este análisis cobra especial relevancia. La compañía utiliza tecnologías modernas como Swift para el desarrollo móvil, Firebase como backend gestionado y GitHub para el control de versiones y la documentación del trabajo. Todas estas herramientas son el resultado directo de los hitos históricos que se analizarán a lo largo de este informe. El objetivo es, por tanto, trazar la línea completa que conecta los orígenes de la web con las prácticas actuales de desarrollo dentro de la industria inmobiliaria tecnológica.

Objetivos del informe

- Investigar las etapas históricas de la evolución de las aplicaciones web, desde sus inicios hasta la actualidad, identificando los hitos clave de cada periodo.
- Analizar cómo esos hitos han influido en las tecnologías y arquitecturas que se utilizan actualmente en el desarrollo web.
- Relacionar esa evolución con el contexto real de una empresa PropTech como PropertyStream Tech y las tecnologías que emplea (Swift, Firebase, GitHub).
- Documentar el análisis de forma estructurada para su publicación y versionado en un repositorio de GitHub.

¿Qué es una aplicación web?

Una aplicación web es un programa de software cuyo cliente se ejecuta dentro de un navegador y cuya lógica puede residir parcial o totalmente en un servidor remoto. A diferencia de las aplicaciones de escritorio tradicionales, que se instalan en el equipo del usuario, las aplicaciones web se acceden a través de una URL y no requieren, en la mayoría de los casos, instalación previa. Esta característica —accesibilidad universal mediante un navegador— es precisamente la que explica el enorme éxito del modelo web.

Toda aplicación web descansa sobre tres pilares fundamentales que se definieron en los primeros años de la Web: el lenguaje de marcado HTML, que estructura el contenido; el protocolo HTTP, que regula la comunicación entre cliente y servidor; y el sistema de direcciones URL, que permite identificar y localizar cada recurso. Sobre esa base mínima se han ido acumulando capas de tecnología que dieron lugar a las plataformas interactivas de hoy.

Etapa 1: La web estática (1990 – 1995)

El origen de la World Wide Web se sitúa en 1989, cuando Tim Berners-Lee, trabajando en el CERN, propuso un sistema de gestión de información basado en documentos enlazados mediante hipervínculos. Su implementación a inicios de los años noventa dio lugar al primer navegador, el primer servidor web y las primeras páginas HTML.

Características principales

En esta etapa las páginas eran completamente estáticas. El servidor almacenaba archivos HTML y los entregaba tal cual al navegador que los solicitaba. No existía personalización: todos los usuarios recibían exactamente el mismo contenido. La interacción se reducía a navegar entre documentos siguiendo enlaces, sin lógica del lado del cliente ni posibilidad de modificar la información mostrada.

Tecnologías de la época

Las piezas tecnológicas eran mínimas pero decisivas: HTML como lenguaje de marcado, HTTP como protocolo de transferencia y la URL como mecanismo de direccionamiento. La aparición de navegadores como Mosaic (1993) y posteriormente Netscape Navigator popularizó el acceso a la Web fuera del ámbito académico, llevándola a un público cada vez más amplio.

> **Hito clave:** la estandarización de HTML, HTTP y la URL estableció los cimientos sobre los que se construiría todo el desarrollo web posterior. Aún hoy, cualquier aplicación web moderna sigue hablando el mismo idioma básico definido en esta etapa.

Etapa 2: La web dinámica del lado del servidor (1995 – 2000)

La limitación evidente de la web estática —entregar siempre el mismo contenido— impulsó el desarrollo de mecanismos para generar páginas de forma dinámica. Así nació la web dinámica del lado del servidor, en la que el contenido se construye en el momento de la petición a partir de bases de datos y de la lógica del programa.

La llegada de los lenguajes del servidor

Tecnologías como CGI (Common Gateway Interface) permitieron ejecutar programas en el servidor para producir HTML dinámico. Poco después surgieron lenguajes y plataformas diseñados específicamente para la web: PHP (1995), Active Server Pages (ASP) de Microsoft y JavaServer Pages (JSP) de Sun. Estas herramientas facilitaron la creación de sitios con contenido personalizado, formularios, sistemas de autenticación y manejo de sesiones.

JavaScript y la interactividad en el cliente

En 1995 Brendan Eich creó JavaScript en Netscape. Aunque inicialmente servía para validaciones sencillas de formularios y pequeños efectos visuales, su aparición fue determinante: por primera vez el navegador podía ejecutar lógica sin depender por completo del servidor. Este lenguaje se convertiría, décadas más tarde, en el motor de las aplicaciones web modernas.

Bases de datos y persistencia

La web dinámica consolidó el uso de bases de datos relacionales como MySQL y PostgreSQL para almacenar y recuperar información de forma estructurada. La combinación de un lenguaje de servidor con una base de datos dio lugar a la arquitectura clásica de tres capas: presentación, lógica de negocio y datos.

> **Hito clave:** la separación entre presentación y datos sentó las bases del modelo cliente-servidor moderno y del concepto de aplicación web como software con estado, capaz de recordar y personalizar la experiencia de cada usuario.

Etapa 3: AJAX y la Web 2.0 (2000 – 2010)

El inicio del nuevo milenio trajo un cambio de paradigma en la experiencia de usuario. Hasta entonces, cada interacción relevante implicaba recargar la página completa, lo que resultaba lento y poco fluido. La técnica AJAX cambió por completo esta dinámica.

La técnica AJAX

AJAX (Asynchronous JavaScript and XML) permitió que el navegador intercambiara datos con el servidor en segundo plano, mediante el objeto XMLHttpRequest, y actualizara únicamente las partes necesarias de la página sin recargarla por completo. Aplicaciones como Gmail (2004) y Google Maps (2005) demostraron que una aplicación web podía ofrecer una experiencia comparable a la de una aplicación de escritorio.

La filosofía Web 2.0

El término Web 2.0 describe la transición de una web de consumo pasivo a una web participativa, donde el usuario genera y comparte contenido. Surgieron blogs, wikis, redes sociales como Facebook y Twitter, y plataformas colaborativas. Las APIs públicas permitieron que los servicios se conectaran entre sí, dando origen a los primeros mashups y a un ecosistema de servicios interconectados.

Consecuencias para el desarrollo

Esta etapa elevó las expectativas de los usuarios respecto a la velocidad y la interactividad. El front-end comenzó a cobrar un peso propio y surgieron las primeras bibliotecas para simplificar el trabajo con JavaScript, como jQuery (2006), que abstraía las diferencias entre navegadores y facilitaba la manipulación del DOM.

> **Hito clave:** el objeto XMLHttpRequest y el nacimiento de las interfaces ricas (RIA) marcaron el inicio de las aplicaciones web verdaderamente interactivas y sentaron las bases del front-end moderno.

Etapa 4: Single Page Applications y frameworks de front-end (2010 – 2016)

El crecimiento de la complejidad de las interfaces hizo que manipular el DOM manualmente con jQuery resultara difícil de mantener en proyectos grandes. La respuesta fueron los frameworks de front-end y el modelo de las Single Page Applications.

El modelo SPA

En una Single Page Application, la aplicación se carga una sola vez y, a partir de ahí, el enrutamiento y el renderizado ocurren en el cliente. En lugar de pedir páginas completas al servidor, la aplicación consume datos a través de APIs y actualiza la interfaz de forma dinámica. Esto produce una experiencia rápida y fluida, muy parecida a la de una aplicación nativa.

Los grandes frameworks

AngularJS (2010) de Google introdujo conceptos como el enlace de datos bidireccional. React (2013) de Facebook popularizó la programación basada en componentes y el DOM virtual, optimizando el rendimiento del renderizado. Vue.js (2014) ofreció una alternativa ligera y accesible. Estos frameworks estructuraron el desarrollo de interfaces complejas y siguen siendo dominantes en la actualidad.

Node.js y JavaScript en el servidor

En 2009, Node.js llevó JavaScript al lado del servidor, permitiendo usar un mismo lenguaje en todo el stack. Junto con el gestor de paquetes npm, dio origen a un ecosistema enorme de módulos reutilizables y consolidó el modelo de APIs REST que intercambian datos en formato JSON.

> **Hito clave:** la separación total entre back-end (una API que expone datos) y front-end (un cliente que los consume) se convirtió en el estándar de arquitectura, habilitando que web y móvil compartieran un mismo servidor.

Etapa 5: Nube, servicios gestionados y "móvil primero" (2016 – actualidad)

La etapa más reciente se caracteriza por la externalización de la infraestructura y por la convergencia entre la web y lo móvil. Los desarrolladores dejaron de administrar servidores físicos para apoyarse en servicios en la nube que escalan automáticamente.

Computación en la nube

Proveedores como Amazon Web Services, Google Cloud y Microsoft Azure ofrecen infraestructura bajo demanda. Los contenedores (Docker), la orquestación (Kubernetes) y las arquitecturas de microservicios permiten construir sistemas distribuidos, resilientes y escalables. El modelo serverless lleva esta idea al extremo: el desarrollador escribe funciones y el proveedor se encarga de ejecutarlas y escalarlas.

Backend as a Service y Firebase

Plataformas como Firebase, de Google, ofrecen Backend as a Service: autenticación de usuarios, bases de datos en tiempo real, almacenamiento de archivos y notificaciones, todo gestionado sin necesidad de administrar servidores. Esto permite que equipos pequeños lancen productos completos y escalables con mucho menos esfuerzo de infraestructura.

Progressive Web Apps y "móvil primero"

Las Progressive Web Apps (PWA) acercan la experiencia web a la de una aplicación nativa: funcionan sin conexión, pueden instalarse en el dispositivo y envían notificaciones. En paralelo, el enfoque "móvil primero" reconoce que la mayoría del tráfico proviene de dispositivos móviles, por lo que el diseño parte de la pantalla pequeña hacia la grande.

> **Hito clave:** la externalización de la infraestructura y la integración continua mediante repositorios como GitHub permiten ciclos de desarrollo rápidos, despliegues automáticos y colaboración distribuida a escala global.

Tabla comparativa de las etapas

Etapa	Periodo	Tecnologías representativas	Aporte principal
Web estática	1990–1995	HTML, HTTP, URL	Documentos enlazados accesibles por navegador

Etapa	Periodo	Tecnologías representativas	Aporte principal
Web dinámica	1995–2000	CGI, PHP, ASP, JSP, MySQL	Contenido personalizado y estado de usuario
AJAX / Web 2.0	2000–2010	XMLHttpRequest, jQuery, APIs	Interactividad sin recargar la página
SPA / Frameworks	2010–2016	Angular, React, Vue, Node.js	Separación total front-end / back-end
Nube / Móvil primero	2016–hoy	Firebase, Docker, PWA, serverless	Infraestructura gestionada y escalable

Influencia de los hitos en las tecnologías actuales

Cada etapa dejó una herencia concreta y acumulativa en el desarrollo web contemporáneo. La web estática aportó el lenguaje común (HTML, HTTP, URL) que todavía usamos. La web dinámica introdujo la separación entre presentación y datos, que evolucionó hasta la arquitectura de APIs que hoy conecta aplicaciones móviles y web con un mismo back-end. La era de AJAX heredó la idea de actualizar la interfaz sin recargar, hoy presente en cualquier aplicación reactiva. Las SPA consolidaron la programación por componentes y el desacople entre cliente y servidor. Finalmente, la nube permitió que el desarrollo se concentrara en el producto y no en la infraestructura.

Esta herencia explica por qué las tecnologías modernas tienen la forma que tienen. React existe porque las interfaces se volvieron demasiado complejas para manipular el DOM a mano. Firebase existe porque administrar servidores resultaba costoso y lento para equipos pequeños. GitHub existe porque el desarrollo colaborativo y distribuido necesitaba un sistema robusto de control de versiones. Ninguna de estas herramientas surgió de forma aislada: todas son respuestas a problemas planteados en etapas anteriores.

Aplicación al caso de PropertyStream Tech

El recorrido histórico anterior se materializa de forma muy directa en las decisiones tecnológicas de PropertyStream Tech, empresa del sector PropTech responsable de las plataformas Smart Listings y RentStream.

Firebase como backend gestionado

El uso de Firebase es consecuencia directa de la etapa de servicios en la nube y Backend as a Service. Gracias a su base de datos en tiempo real, Smart Listings puede reflejar de inmediato los cambios en los listados de propiedades —precio, disponibilidad, fotografías— sin que el usuario deba recargar. La autenticación gestionada de Firebase simplifica el registro y el inicio de sesión, y sus notificaciones mantienen informados a arrendadores e inquilinos en RentStream. Todo ello sin que la empresa deba administrar servidores propios.

Swift y el enfoque "móvil primero"

El uso de Swift para el cliente móvil responde al enfoque "móvil primero" heredado de la última etapa evolutiva. Dado que gran parte de los usuarios de plataformas inmobiliarias consulta propiedades desde el teléfono, una aplicación nativa rápida y fluida ofrece una ventaja competitiva clara. Swift permite construir ese cliente aprovechando al máximo las capacidades del dispositivo, mientras consume los datos expuestos por el back-end, siguiendo el modelo de separación cliente-servidor consolidado en la etapa de las SPA.

GitHub como columna vertebral del trabajo

GitHub materializa las prácticas de integración continua, control de versiones y documentación surgidas con la industrialización del software. En PropertyStream Tech, el equipo documenta su análisis, versiona el código de Smart Listings y RentStream, revisa cambios mediante pull requests y mantiene un historial completo del proyecto. Es, además, el lugar donde se publica el informe de este mismo análisis, cerrando el ciclo entre la teoría y la práctica.

> **Relación directa:** Smart Listings aprovecha la sincronización en tiempo real (herencia de AJAX y la nube), RentStream se apoya en servicios gestionados (Firebase) y clientes móviles nativos (Swift), y ambos proyectos se documentan y versionan en GitHub. Los hitos históricos no son teoría aislada: están presentes en cada decisión técnica de la empresa.

Planificación de un ambiente de desarrollo moderno

Comprender la evolución de la web también orienta la preparación de un ambiente de desarrollo adecuado. Un entorno actual suele incluir: un editor de código con soporte para el lenguaje y el framework elegidos; un sistema de control de versiones basado en Git y alojado en GitHub; un gestor de dependencias (como npm o Swift Package Manager); herramientas de automatización de pruebas; y una canalización de integración y despliegue continuos. Elegir cada una de estas piezas con conocimiento de su origen y propósito evita decisiones arbitrarias y facilita el mantenimiento del proyecto a largo plazo.

- **Control de versiones:** Git + GitHub para historial, ramas y colaboración.
- **Back-end:** servicios gestionados como Firebase cuando se prioriza la velocidad de desarrollo.
- **Front-end / móvil:** frameworks de componentes (React, Vue) o lenguajes nativos (Swift) según el objetivo.
- **Automatización:** pruebas e integración continua para asegurar calidad en cada cambio.

Conclusión

La evolución de las aplicaciones web es, ante todo, una historia de abstracción creciente: de simples documentos estáticos a plataformas distribuidas, interactivas y alojadas en la nube. Cada hito resolvió una limitación de la etapa anterior —la falta de dinamismo, la lentitud de las recargas, la complejidad del front-end, el costo de la infraestructura— y, al hacerlo, abrió un abanico de nuevas posibilidades.

Entender este recorrido permite preparar un ambiente de desarrollo sólido y, sobre todo, justificar de manera razonada las tecnologías modernas. En el caso de PropertyStream Tech, herramientas como Swift, Firebase y GitHub no son elecciones caprichosas, sino el resultado natural de décadas

de avances acumulados que hoy permiten a la empresa ofrecer plataformas competitivas en la industria PropTech. El estudiante que comprende esta línea de tiempo está mejor preparado para tomar decisiones técnicas con fundamento y para adaptarse a las próximas etapas de una evolución que no se detiene.

Referencias

- Berners-Lee, T. (1989). *Information Management: A Proposal*. CERN.
- Mozilla Developer Network. *Historia de la Web y tecnologías web*. MDN Web Docs.
- Garrett, J. J. (2005). *Ajax: A New Approach to Web Applications*. Adaptive Path.
- Firebase. *Documentación oficial de Firebase*. Google.
- Apple Inc. *The Swift Programming Language*. Apple Developer Documentation.
- GitHub. *Guías de documentación y control de versiones*. GitHub Docs.
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Tesis doctoral, UC Irvine.